CLEAN KIGALI

Entity relationship model

The database behind the community sanitation reporting service: its tables, how they connect, what enforces each connection, and where it needs strengthening.

6	59	15	11	0
tables	columns	indexes	relationships	foreign keys declared

The main thing to know

All 11 relationships exist only in application code or stored values. The database itself enforces none of them, so the code carries the whole job. This review found no faults in that code. The biggest risk is not technical: a leader only sees a report when the cell name typed on it matches theirs exactly.

Schema read from the live database `umuganda` on MariaDB 10.11.19, 2 November 2026. Working copy on commit a8b4321, with uncommitted changes.

Contents

Conceptual model 2

Physical ERD 3

Relationship catalogue 4

Data dictionary: reports 5

Data dictionary: events and accounts 6

Data dictionary: audit, sessions, rate limits 7

Report lifecycle 8

Value lists 9

Indexes and write paths 10

Findings 11

Proposed schema changes 12

Appendix: live table definitions 13

To see the structure

Start with the business view on page 2, then every table and column on page 3. Page 3 is A3, so zoom in or print it large.

To check a connection or a column

Page 4 gives the rule and source line behind each line in the diagram. Pages 5 to 7 explain each column and who can see it.

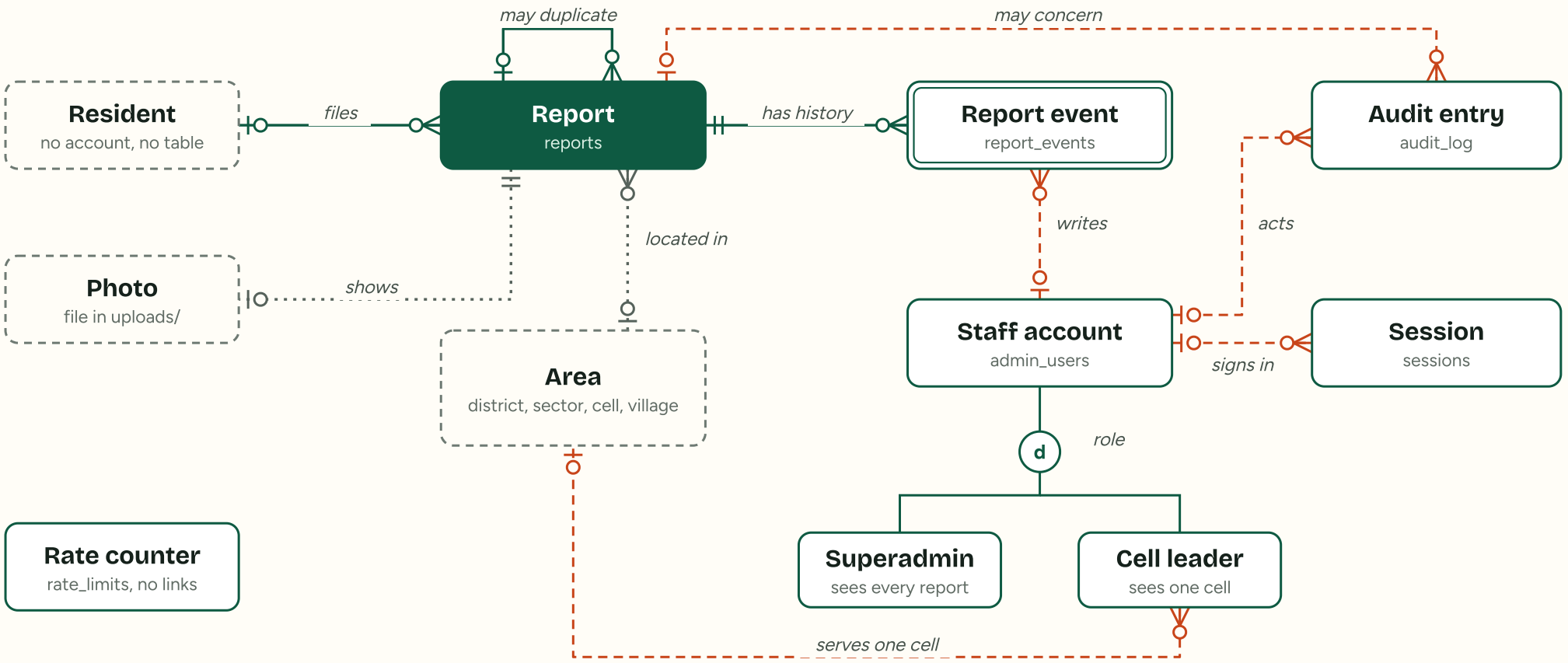
To plan what comes next

Page 11 ranks seven weaknesses. Page 12 has SQL for three of them, tested but not applied.

BUSINESS VIEW

Conceptual model

Report sits at the centre. Everything else records what happened to a report, or who is allowed to act on it.



Reading this page

Solid box: has its own table.
Dashed box: real in the business, but has no table.
Double border: weak entity. An event cannot exist without its report.
Circle d: disjoint subtypes. Each account is exactly one role.

Entities with no table

Resident: anonymous by design. Name and contact are optional fields on the report, so two reports cannot be tied to one person.
Photo: a file on disk, named by the report.
Area: repeated as text on each row.

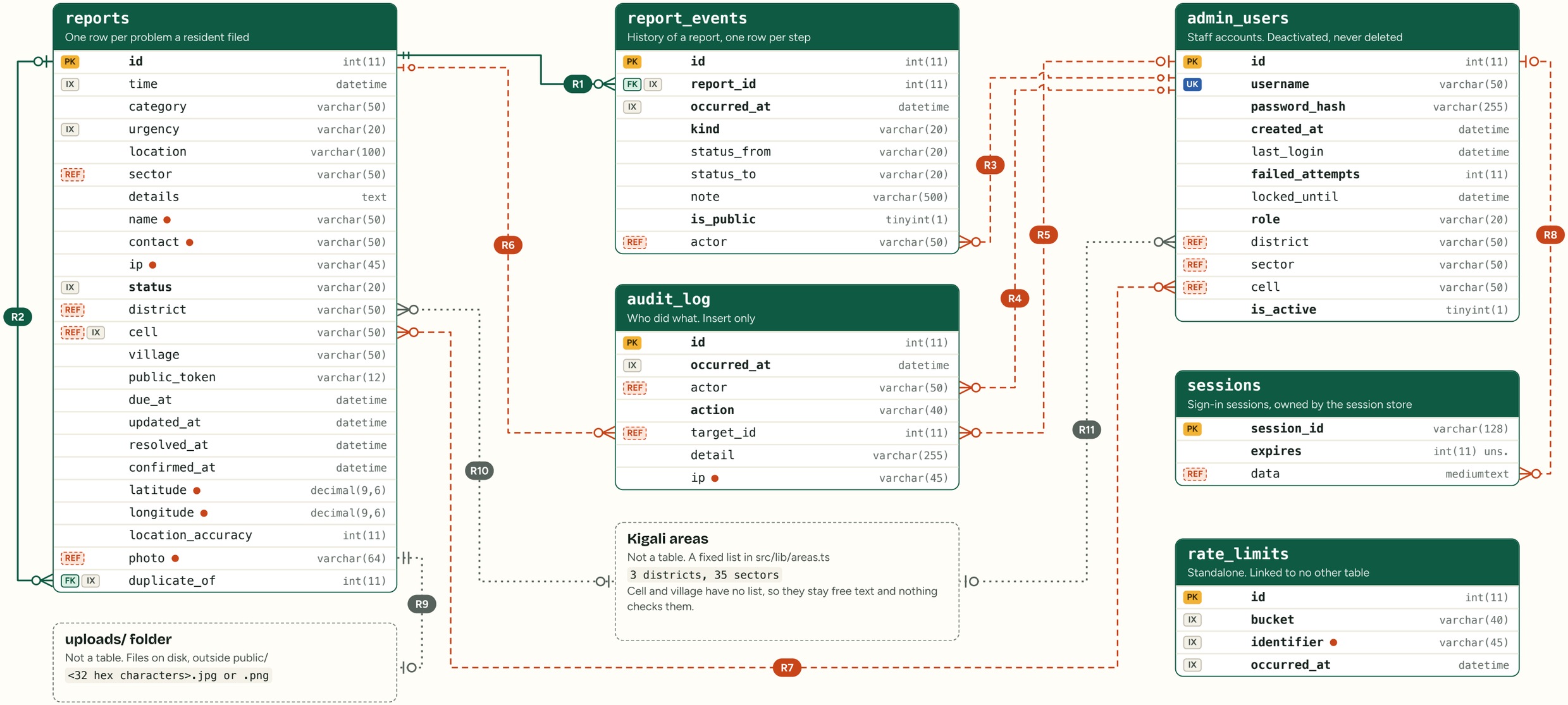
Derived, never stored

Reference: CK-id-token.
Service state: met, missed, on track, at risk or overdue.
Priority score: computed for the queue.
Linked count: reports pointing at this one.
Created step: tracker builds it from reports.time.

AS BUILT

Physical ERD

Every table and column in the live database. **None of these lines is a database constraint:** the schema declares 0 foreign keys.



Relationships

- **R1** **reports to report_events**
  1 to 0..N, on report_id
- **R2** **reports to reports**
  original 0..1 to 0..N duplicates, on duplicate_of
- **R3** **admin_users to report_events**
  0..1 to 0..N, username stored in actor
- **R4** **admin_users to audit_log**
  0..1 to 0..N, username stored in actor
- **R5** **admin_users to audit_log**
  0..1 to 0..N, target_id for account actions
- **R6** **reports to audit_log**
  0..1 to 0..N, target_id for report actions
- **R7** **reports with admin_users**
  N to N, matched on the same cell text
- **R8** **admin_users to sessions**
  0..1 to 0..N, adminId inside the JSON in data
- **R9** **reports to photo file**
  1 to 0..1, file name stored in photo
- **R10** **reports to Kigali areas**
  N to 0..1, on district and sector
- **R11** **admin_users to Kigali areas**
  N to 0..1, on district and sector

Notation

Logical foreign key. Same id type, kept correct by application code.

Reference by value. Text, an untyped id or JSON. Nothing guarantees a match.

Outside the database. Points at a file or a list in code.

exactly one · zero or one · zero or many

PK primary · UK unique · IX indexed · FK logical key · REF by value

bold name NOT NULL · ● personal data

EVERY LINE, WITH EVIDENCE

Relationship catalogue

The model has 11 relationships. Two link on real ids, six rely on stored values that nothing checks, and three point outside the database.

	RELATIONSHIP	LINK	KEPT CORRECT BY	WHEN THE PARENT IS REMOVED	EVIDENCE (SRC/)
R1	reports 1 to 0..N report_events	`report_events.report_id = reports.id`	Application, in a transaction	Events are deleted first, in the same transaction	`services/reports.ts:580`
R2	reports 0..1 to 0..N reports	`reports.duplicate_of = reports.id`	Application. Always points at the first report, so links never chain	Linked reports get NULL and stand alone	`services/reports.ts:582`
R3	admin_users 0..1 to 0..N report_events	`report_events.actor = username`	Nothing. Copied as text, or the literal reporter or system	Accounts are never deleted, so the name stays valid	`services/reports.ts:193`
R4	admin_users 0..1 to 0..N audit_log	`audit_log.actor = username`	Nothing. Copied as text, or a script name	Same as R3	`services/audit.ts:35`
R5	admin_users 0..1 to 0..N audit_log	`audit_log.target_id = admin_users.id`	The action says the type: account_activated, account_deactivated, password_changed	Accounts are never deleted	`services/admins.ts:357, 460`
R6	reports 0..1 to 0..N audit_log	`audit_log.target_id = reports.id`	The action says the type: status_change, report_note, report_deleted	Audit row stays and points at nothing, on purpose. It is the proof of the deletion	`routes/admin.ts:451, 655`
R7	reports N to N admin_users	`reports.cell = admin_users.cell`	Every staff query adds this as a WHERE clause	Reports stay. With no active leader, only a superadmin sees them	`services/reports.ts:301`
R8	admin_users 0..1 to 0..N sessions	`sessions.data $.adminId`	Application. Sessions before sign-in carry no adminId	Deactivating an account or changing a password deletes its sessions	`services/admins.ts:350, 474`
R9	reports 1 to 0..1 photo file	`reports.photo = file name`	Application. Name format is checked before any disk access	File is removed after the commit. A failure is logged	`routes/admin.ts:654`
R10	reports N to 0..1 Kigali areas	`reports.district, sector`	Validated as a pair on submission	The list is fixed in code	`lib/validate.ts:78`
R11	admin_users N to 0..1 Kigali areas	`admin_users.district, sector`	Validated when a leader is created	The list is fixed in code	`lib/areas.ts`

rate_limits has no relationships. It counts actions per IP address and shares only the kind of value with `reports.ip` and `audit_log.ip`. They are never joined. **`public_token` is not an index or a link:** the row is found by id, then the token is compared in constant time.

COLUMNS

Data dictionary: reports

Seen by **P** anyone, on the public feed **R** whoever holds the reference, on the tracker **S** staff. A leader sees only their own cell, a superadmin sees all ● personal data **bold** NOT NULL

COLUMN	TYPE	NULL	DEFAULT	KEY	MEANING	P	R	S
id	int(11)		auto	PK	Row identity. First half of the reference CK-id-token.	●	●	●
time	datetime	yes		IX	When it was filed, set by the server. Null only on legacy rows.	●	●	●
category	varchar(50)	yes			One of 5 allowed values, checked in the app.	●	●	●
urgency	varchar(20)	yes		IX	low, medium or high. Sets the length of the service target.	●	●	●
location	varchar(100)	yes			Landmark in the reporter's words, up to 100 characters.	●	●	●
sector	varchar(50)	yes		REF	Checked as a valid pair with district. Required on new reports.	●	●	●
details	text	yes			Free text, up to 2000 characters. Shown on the open feed.	●	●	●
name ●	varchar(50)	yes			Reporter's name. Optional.			●
contact ●	varchar(50)	yes			Reporter's phone or email. Optional.			●
ip ●	varchar(45)	yes			Address the report came from. Staff panel only, not in the CSV.			●
status	varchar(20)		pending	IX	Lifecycle state, see Lifecycle page. Legacy rows may hold an empty value.	●	●	●
district	varchar(50)	yes		REF	Checked against the 3 Kigali districts. Required on new reports.	●		●
cell	varchar(50)	yes		IX REF	Free text, required. Decides which leader sees the report.	●		●
village	varchar(50)	yes			Free text. Optional.	●		●
public_token	varchar(12)	yes			6 random characters. A tracker lookup must present it.		●	●
due_at	datetime	yes			time + service target. Reset when the reporter disputes a closure.		●	●
updated_at	datetime	yes			Last change to the row.			●
resolved_at	datetime	yes			Set on closing. Cleared when the report goes back to work.			●
confirmed_at	datetime	yes			When the reporter confirmed the fix. The tracker shows yes or no.		●	●
latitude ●	decimal(9,6)	yes			Optional GPS from the reporter's device.			●
longitude ●	decimal(9,6)	yes			Optional GPS from the reporter's device.			●
location_accuracy	int(11)	yes			GPS accuracy in metres.			●
photo ●	varchar(64)	yes		REF	Random file name under uploads/. Metadata is stripped on upload.		●	●
duplicate_of	int(11)	yes		IX FK	The original report, when this looks like the same problem.			●

The CSV export carries every staff field except `ip`, `photo`, `updated_at` and `location_accuracy`. The token goes out as part of the reference.

COLUMNS

Data dictionary: events and accounts

Seen by **P** anyone, on the public feed **R** whoever holds the reference, on the tracker **S** staff. A leader sees only their own cell, a superadmin sees all ● personal data **bold** NOT NULL

report_events

COLUMN	TYPE	NULL	DEFAULT	KEY	MEANING	P	R	S
id	int(11)		auto	PK	Row identity. Breaks ties in time order.			●
report_id	int(11)			IX FK	The report this step belongs to.			●
occurred_at	datetime			IX	When it happened.		●	●
kind	varchar(20)				status, note, confirmed, reopened or duplicate.		●	●
status_from	varchar(20)	yes			Status before the move. Null for notes.			●
status_to	varchar(20)	yes			Status after the move. Null for notes.		●	●
note	varchar(500)	yes			Up to 500 characters. The tracker shows it only on public rows.		●	●
is_public	tinyint(1)		1		1 shows the row on the tracker, 0 keeps it for staff.			●
actor	varchar(50)	yes		REF	A staff username, or the literal reporter or system.			●

admin_users

COLUMN	TYPE	NULL	DEFAULT	KEY	MEANING
id	int(11)		auto	PK	Row identity. Held in the session after sign-in.
username	varchar(50)			UK	3 to 50 characters. Written into events and the audit log.
password_hash	varchar(255)				bcrypt digest, cost 12. Never leaves the server.
created_at	datetime				When the account was created.
last_login	datetime	yes			Last successful sign-in.
failed_attempts	int(11)		0		Consecutive failures. 5 locks the account.
locked_until	datetime	yes			Lockout end, 15 minutes after the 5th failure.
role	varchar(20)		leader		superadmin or leader. Default is leader.
district	varchar(50)	yes		REF	Leader's district. Null for a superadmin.
sector	varchar(50)	yes		REF	Leader's sector. Null for a superadmin.
cell	varchar(50)	yes		REF	Leader's scope: the only reports they can see. Null for a superadmin.
is_active	tinyint(1)		1		0 blocks sign-in and removes the account's sessions.

Only a superadmin can open the accounts page. `password_hash` and `failed_attempts` are never displayed.

COLUMNS

Data dictionary: audit, sessions, rate limits

● personal data **bold** NOT NULL

audit_log

COLUMN	TYPE	NULL	DEFAULT	KEY	MEANING
id	int(11)		auto	PK	Row identity.
occurred_at	datetime			IX	When it happened.
actor	varchar(50)	yes		REF	A staff username, or seed-script or set-password-script.
action	varchar(40)				One of 12 values, see Value lists page.
target_id	int(11)	yes		REF	A report id or an account id, depending on action. Often null.
detail	varchar(255)	yes			Short description, up to 255 characters.
ip ●	varchar(45)	yes			Address the action came from.

sessions

COLUMN	TYPE	NULL	DEFAULT	KEY	MEANING
session_id	varchar(128)			PK	Random id, also held in the signed cookie.
expires	int(11) unsigned				Unix time in seconds. Expired rows are swept every 15 minutes.
data	mediumtext	yes		REF	JSON text. Holds adminId, role, cell, CSRF token and timestamps.

rate_limits

COLUMN	TYPE	NULL	DEFAULT	KEY	MEANING
id	int(11)		auto	PK	Row identity.
bucket	varchar(40)			IX	submit, track or login.
identifier ●	varchar(45)			IX	Client IP address.
occurred_at	datetime			IX	One row per counted action. Rows older than a day get swept.

No screen in the app shows these three tables.

HOW THE STATUS COLUMN MAY CHANGE

Report lifecycle

Allowed moves

From \ To	pending	in_progress	done	notdone	reopened
pending		Staff	Staff	Staff	
in_progress	Staff		Staff	Staff	
done		Staff			Reporter
notdone		Staff			Reporter
reopened		Staff	Staff	Staff	

Generated from `canTransition()` in `src/lib/workflow.ts`. Each move is checked after locking the row with `SELECT ... FOR UPDATE`.

Service targets

URGENCY	TARGET FROM FILING
high	24 hours
medium	72 hours (3 days)
low	168 hours (7 days)

A report is at risk in the last quarter of its window. The reporter may dispute a closure for 14 days.

What each move writes

MOVE	ON REPORTS	ON REPORT_EVENTS
Filed	`status` pending, `due_at`, `public_token`, maybe `duplicate_of`	No row for this report. A duplicate adds a private system row on the original
Staff closes	`resolved_at` = now	status, public
Staff reopens or starts	`resolved_at` and `confirmed_at` cleared	status, public
Staff adds a note	`updated_at` only	note, public or private
Private note with a move	As for the move	status, public, plus a separate private note row
Reporter confirms	`confirmed_at` = now. Status unchanged	confirmed, public
Reporter disputes	`status` reopened, `resolved_at` cleared, new `due_at`	reopened, public

Rules the code keeps

Closed means stamped. New rows have `resolved_at` set exactly while status is done or notdone.

Confirmation ends disputes. Once `confirmed_at` is set, the reporter has no further action.

Links have one level. `duplicate_of` is set only at filing, and always at an original.

Nothing is merged. A duplicate keeps its own reference and row.

ALLOWED VALUES, CHECKED IN CODE

Value lists

report_events.kind

VALUE	WRITTEN BY	VISIBLE ON TRACKER
status	Staff username	Always
note	Staff username	Only when marked public
confirmed	reporter	Always
reopened	reporter	Always
duplicate	system	Never

Short lists

COLUMN	VALUES
`reports.status`	pending, in_progress, done, notdone, reopened
`reports.urgency`	low, medium, high
`reports.category`	full-bin, blocked-drain, illegal-dumping, litter, other
`admin_users.role`	superadmin, leader
`reports.district`	Gasabo (15 sectors), Kicukiro (10 sectors), Nyarugenge (10 sectors)

rate_limits.bucket

BUCKET	LIMIT PER IP	COUNTS
submit	5 per 60 minutes	Reports filed
track	30 per 10 minutes	Tracker lookups
login	15 per 15 minutes	Failed sign-ins only

audit_log.action

ACTION	TARGET_ID POINTS AT
status_change	reports.id
report_note	reports.id
report_deleted	reports.id, now gone
account_activated	admin_users.id
account_deactivated	admin_users.id
password_changed	admin_users.id
admin_created	Null. The name is in detail
reports_exported	Null
login_success	Null
login_failed	Null. Known accounts only
login_locked	Null
logout	Null

A failed sign-in with an unknown username writes no audit row, but it still counts against the IP limit.

Session and lockout limits

LIMIT	VALUE
Idle sign-out	30 minutes
Absolute sign-out	12 hours
Account lockout	5 failures, 15 minutes
Duplicate match	7 days, within 150 m or the same cell

PHYSICAL BEHAVIOUR

Indexes and write paths

All 15 indexes

TABLE	INDEX	COLUMNS	USED BY	QUERY PLAN
reports	PRIMARY	id	Tracker, row lock	const, 1 row
	idx_time	time	Public feed, duplicate search	8 rows; 783 rows
	idx_cell	cell	Leader queue	ref, 125 rows
	idx_duplicate_of	duplicate_of	Linked count	covering, 5 rows
	idx_status	status	Rarely chosen	full scan on done
	idx_urgency	urgency	No query	unused
report_events	PRIMARY	id	Row identity	
	idx_report_time	report_id, occurred_at	Tracker timeline	ref, 2 rows
admin_users	PRIMARY	id	Password change, deactivate	
	uniq_username	username	Sign-in, uniqueness	
audit_log	PRIMARY	id	Row identity	
	idx_occurred	occurred_at	No app query reads this table	unused
rate_limits	PRIMARY	id	Row identity	
	idx_bucket_id_time	bucket, identifier, occurred_at	Rate check	covering, 25 rows
sessions	PRIMARY	session_id	Every signed-in request	

Plans are MariaDB `EXPLAIN` estimates on a throwaway copy with 20,000 synthetic reports, using the app's real query shapes. They show which index is picked, not production timings.

Multi-step writes

ACTION	STEPS
File a report	Save photo. In a transaction: read open candidates, insert report, add duplicate event. Remove photo if the insert fails.
Change status	In a transaction: lock row, check move, update report, insert event or events. Audit row follows.
Reporter feedback	In a transaction: lock row, check rules, update report, insert event.
Delete a report	In a transaction: lock row, delete events, clear links, delete report. Photo and audit row follow.
Deactivate account	Update account, delete sessions, write audit row. No transaction, each step separate.

An audit row is written after its action commits. If that write fails, the error is logged and the action still stands.

Not indexed, and no need to be

COLUMN	WHY
`public_token`	Never searched. The row is found by id, then the token is compared.
`sessions.data`	Scanned with JSON_EXTRACT, only when an account is deactivated or its password changes.

Cleaning itself up

TABLE	HOW
`sessions`	The store sweeps expired rows every 15 minutes.
`rate_limits`	About 2% of writes delete rows older than a day.
`audit_log`	Nothing. Grows forever.

WHAT THE MODEL DOES NOT PROTECT YET

Findings

The schema works and is consistent. These seven points are where it relies on code or on people typing correctly.

SEVERITY	FINDING	WHAT HAPPENS	RECOMMENDATION
High	**Free text decides who sees a report**	Leader scope is an exact match on the typed cell name. Case and trailing spaces are ignored, but spelling is not, so Kamatamu never matches Kamutamu. Tested on the database. A misspelt cell, or a cell with no leader, reaches only the superadmin, and nothing flags it.	Add a superadmin view of reports whose cell has no active leader now. Add a cells table once a verified list exists.
High	**Personal data is kept forever**	Reporter name, contact, IP, GPS and photo have no deletion rule. SECURITY.md lists this as open. audit_log.ip also never expires.	Decide a retention period, then blank these columns on closed reports past that age.
Medium	**No foreign keys in the database**	Application code handles every current path correctly, and the dev database has 0 orphans, though it holds only 1 report. Manual SQL or a future code path could still create orphans without any error.	Add the 2 constraints on the next page. Check for orphans first, because the constraint is refused if any exist.
Medium	**No value checks on status, kind or role**	These are plain VARCHAR columns, so only application code keeps them valid. Legacy rows may hold an empty status.	Clean legacy rows, then add the 3 CHECK constraints on the next page.
Medium	**The audit log cannot be read in the app**	No application query reads audit_log, so staff need direct database access to use it. It has no retention rule, and target_id has no column saying what kind of row it points at.	Add a superadmin screen and a target_type column.
Low	**Indexes do not match the queries**	Duplicate matching runs on every submission without its best index, and resolution charts can scan the whole table. No query filters on urgency alone, so idx_urgency does no useful work.	Add 2 composite indexes and drop 1, as tested on the next page.
Low	**Timestamps carry no time zone**	DATETIME values hold the app server's local time and no TZ is set. On a host running UTC, times and chart day boundaries would be 2 hours off Kigali time.	Set TZ=Africa/Kigali in the deployment.

Severity is a judgement of the effect on residents and staff. The sessions table uses a different collation (`utf8mb4_general_ci`) from the others, which is harmless because it is never joined.

TESTED, NOT APPLIED

Proposed schema changes

```
-- Check first: both must return 0
SELECT COUNT(*) FROM report_events e
  LEFT JOIN reports r ON r.id = e.report_id
 WHERE r.id IS NULL;
SELECT COUNT(*) FROM reports a
  LEFT JOIN reports b ON b.id = a.duplicate_of
 WHERE a.duplicate_of IS NOT NULL AND b.id IS NULL;

-- Foreign keys for R1 and R2
ALTER TABLE report_events
  ADD CONSTRAINT fk_events_report
  FOREIGN KEY (report_id) REFERENCES reports(id)
  ON DELETE CASCADE;
ALTER TABLE reports
  ADD CONSTRAINT fk_reports_duplicate
  FOREIGN KEY (duplicate_of) REFERENCES reports(id)
  ON DELETE SET NULL;
```

```
-- Clean legacy values, then check values
UPDATE reports SET status = 'pending'
 WHERE status NOT IN ('pending','in_progress','done','notdone','reopened');
ALTER TABLE reports ADD CONSTRAINT chk_status CHECK
  (status IN ('pending','in_progress','done','notdone','reopened'));
ALTER TABLE report_events ADD CONSTRAINT chk_kind CHECK
  (kind IN ('status','note','confirmed','reopened','duplicate'));
ALTER TABLE admin_users ADD CONSTRAINT chk_role CHECK
  (role IN ('superadmin','leader'));

-- Indexes that match the queries
ALTER TABLE reports
  ADD INDEX idx_category_time (category, time),
  ADD INDEX idx_status_resolved (status, resolved_at),
  DROP INDEX idx_urgency;
```

Test results on a throwaway copy with 20,000 reports

TEST	RESULT
Delete a report that has 2 events and 1 linked report	Events went from 2 to 0. The linked report's duplicate_of became NULL
Insert an event for a report that does not exist	Refused with error 1452
Set status to an invalid value	Refused with error 4025
Add the foreign key while orphan rows exist	Refused with error 1452, which is why the check comes first
Run the app's delete statements with the constraints on	Completed, the row is gone
Duplicate search plan	Estimated rows fell from 783 to 112 with idx_category_time
Resolution chart plan	Full scan of 20,046 rows became a range of 1,562 with idx_status_resolved

Nothing here has been applied to your database. The app has no migration tool, so these belong in `src/db.ts`, the same way earlier columns were added. The full app has not been run against them.

FROM MARIADB-DUMP --NO-DATA

Appendix: live table definitions

```
CREATE TABLE `reports` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `time` datetime DEFAULT NULL,
  `category` varchar(50) DEFAULT NULL,
  `urgency` varchar(20) DEFAULT NULL,
  `location` varchar(100) DEFAULT NULL,
  `sector` varchar(50) DEFAULT NULL,
  `details` text DEFAULT NULL,
  `name` varchar(50) DEFAULT NULL,
  `contact` varchar(50) DEFAULT NULL,
  `ip` varchar(45) DEFAULT NULL,
  `status` varchar(20) NOT NULL DEFAULT 'pending',
  `district` varchar(50) DEFAULT NULL,
  `cell` varchar(50) DEFAULT NULL,
  `village` varchar(50) DEFAULT NULL,
  `public_token` varchar(12) DEFAULT NULL,
  `due_at` datetime DEFAULT NULL,
  `updated_at` datetime DEFAULT NULL,
  `resolved_at` datetime DEFAULT NULL,
  `confirmed_at` datetime DEFAULT NULL,
  `latitude` decimal(9,6) DEFAULT NULL,
  `longitude` decimal(9,6) DEFAULT NULL,
  `location_accuracy` int(11) DEFAULT NULL,
  `photo` varchar(64) DEFAULT NULL,
  `duplicate_of` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_status` (`status`),
  KEY `idx_urgency` (`urgency`),
  KEY `idx_time` (`time`),
  KEY `idx_cell` (`cell`),
  KEY `idx_duplicate_of` (`duplicate_of`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `report_events` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `report_id` int(11) NOT NULL,
  `occurred_at` datetime NOT NULL,
  `kind` varchar(20) NOT NULL,
  `status_from` varchar(20) DEFAULT NULL,
  `status_to` varchar(20) DEFAULT NULL,
  `note` varchar(500) DEFAULT NULL,
  `is_public` tinyint(1) NOT NULL DEFAULT 1,
  `actor` varchar(50) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_report_time` (`report_id`,`occurred_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `admin_users` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `username` varchar(50) NOT NULL,
  `password_hash` varchar(255) NOT NULL,
  `created_at` datetime NOT NULL,
  `last_login` datetime DEFAULT NULL,
  `failed_attempts` int(11) NOT NULL DEFAULT 0,
  `locked_until` datetime DEFAULT NULL,
  `role` varchar(20) NOT NULL DEFAULT 'leader',
  `district` varchar(50) DEFAULT NULL,
  `sector` varchar(50) DEFAULT NULL,
  `cell` varchar(50) DEFAULT NULL,
  `is_active` tinyint(1) NOT NULL DEFAULT 1,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uniq_username` (`username`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `audit_log` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `occurred_at` datetime NOT NULL,
  `actor` varchar(50) DEFAULT NULL,
  `action` varchar(40) NOT NULL,
  `target_id` int(11) DEFAULT NULL,
  `detail` varchar(255) DEFAULT NULL,
  `ip` varchar(45) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_occurred` (`occurred_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `sessions` (
  `session_id` varchar(128) CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT NULL,
  `expires` int(11) unsigned NOT NULL,
  `data` mediumtext CHARACTER SET utf8mb4 COLLATE utf8mb4_bin DEFAULT NULL,
  PRIMARY KEY (`session_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

CREATE TABLE `rate_limits` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `bucket` varchar(40) NOT NULL,
  `identifier` varchar(45) NOT NULL,
  `occurred_at` datetime NOT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_bucket_id_time` (`bucket`,`identifier`,`occurred_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```